

UE Intelligence Artificielle - Polytech Lyon

Partie 3: Apprentissage profond par renforcement

CM7: Approches basées politique

Laëtitia Matignon
laetitia.matignon@univ-lyon1.fr
Equipe SyCoSMA@LIRIS

- 1 Approches basées valeurs versus basées politiques
- 2 Implémentation des différentes politiques
- 3 Formalisation du problème de recherche de politique
- 4 Recherche directe de π
- 5 Approches par gradient de π
- 6 Conclusion

- 1 Approches basées valeurs versus basées politiques
- 2 Implémentation des différentes politiques
- 3 Formalisation du problème de recherche de politique
- 4 Recherche directe de π
- 5 Approches par gradient de π
- 6 Conclusion

Approches basées valeurs

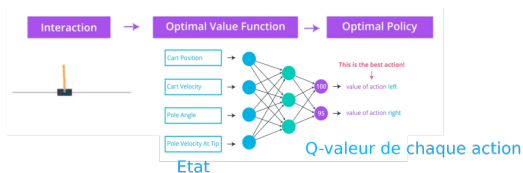
Objectif de l'agent en RL

Trouver la politique optimale $\pi^* = \operatorname{argmax}_{\pi} E_{\pi} [\sum_{t=0}^{\infty} \gamma^t r_{t+1}]$ qui renvoie dans chaque état l'action qui maximise les récompenses cumulées espérées.

Approche indirecte - basées valeurs (par ex. DQN) :

- apprendre une fonction de valeur (Q , tabulaire ou paramétrée par ex. NN)
- déduire π^* de cette fonction de valeur :

$$\pi^*(s) = \operatorname{argmax}_{a \in A} Q(s, a)$$



Approches basées valeurs

Avantages des approches basées valeurs (type DQN)

Convergence rapide de l'apprentissage car :

- Efficacité en données : utilisation de *replays buffers* pour réutiliser d'anciens échantillons
- Apprentissage stable : variance faible avec l'utilisation des Q valeurs pour évaluer la somme des récompenses futures espérées (pas de Monte-Carlo)

Limites des approches basées valeurs (type DQN)

- Actions continues impossibles
- Stratégie d'exploration indispensable (par ex. ϵ -greedy)
- Apprentissage erratique (divergence possible) dans certains cas : changement brusque de la politique car une modification mineure de Q peut changer radicalement l'action gloutonne

Approches basées valeurs

Avantages des approches basées valeurs (type DQN)

Convergence rapide de l'apprentissage car :

- Efficacité en données : utilisation de *replays buffers* pour réutiliser d'anciens échantillons
- Apprentissage stable : variance faible avec l'utilisation des Q valeurs pour évaluer la somme des récompenses futures espérées (pas de Monte-Carlo)

Limites des approches basées valeurs (type DQN)

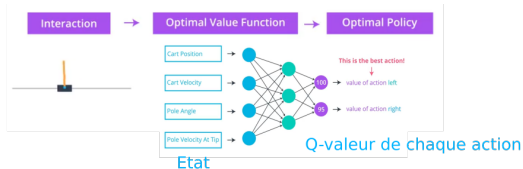
- Actions continues impossibles
- Stratégie d'exploration indispensable (par ex. ϵ -greedy)
- Apprentissage erratique (divergence possible) dans certains cas : changement brusque de la politique car une modification mineure de Q peut changer radicalement l'action gloutonne

2 approches pour trouver π^*

Approche indirecte - basée valeur (par ex. DQN) :

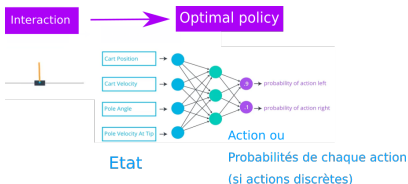
- apprendre une fonction de valeur Q
- déduire π^* de cette fonction de valeur :

$$\pi^*(s) = \operatorname{argmax}_{a \in A} Q(s, a)$$



Approche directe - basée politique :

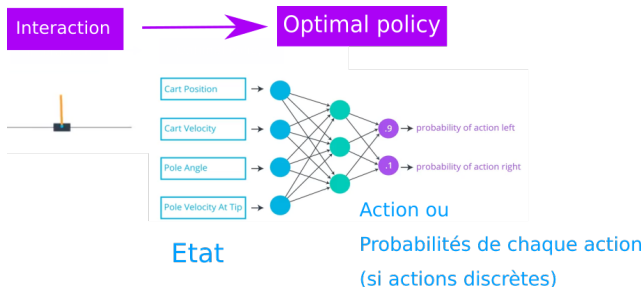
- représentation **explicite** de la politique : fonction paramétrée π_θ
- **recherche directe de π_θ^* dans l'espace des politiques** (sans fonction intermédiaire)



Politique paramétrée

La politique π détermine l'action à prendre dans un état (comportement de l'agent).

On note π_θ une **politique paramétrée** par un ensemble de paramètres θ (par ex. NN), que l'on modifie pour ajuster le comportement de l'agent.



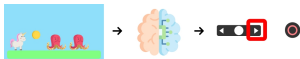
Politique déterministe et stochastique

Une politique peut être **déterministe** ou **stochastique**.

- Une politique **déterministe** renvoie pour un état s l'**action** à faire dans s :

$$\pi : S \rightarrow A$$

$$a = \pi(s)$$



$$\text{State } s_0 \rightarrow \pi(s_0) \rightarrow a_0 = \text{Right}$$

Avec cette politique déterministe, l'action exécutée dans s_0 sera toujours Right !

- Une politique **stochastique** renvoie pour un état s une **distribution de probabilités** sur l'ensemble des actions possibles dans s :

$$\pi : S, A \rightarrow [0, 1]$$

$$\pi(a|s) = P_r[a|s]$$

Dans l'état s , on choisit une action selon la distribution de probabilités $P_r[a|s]$, on note $a_t \sim \pi(\cdot|s_t)$

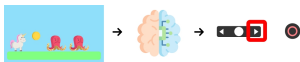
Politique déterministe et stochastique

Une politique peut être **déterministe** ou **stochastique**.

- Une politique **déterministe** renvoie pour un état s l'**action** à faire dans s :

$$\pi : S \rightarrow A$$

$$a = \pi(s)$$



$$\text{State } s_0 \rightarrow \pi(s_0) \rightarrow a_0 = \text{Right}$$

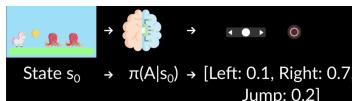
Avec cette politique déterministe, l'action exécutée dans s_0 sera toujours Right !

- Une politique **stochastique** renvoie pour un état s une **distribution de probabilités** sur l'ensemble des actions possibles dans s :

$$\pi : S, A \rightarrow [0, 1]$$

$$\pi(a|s) = P_r[a|s]$$

Dans l'état s , on choisit une action selon la distribution de probabilités $P_r[a|s]$, on note $a_t \sim \pi(\cdot|s_t)$



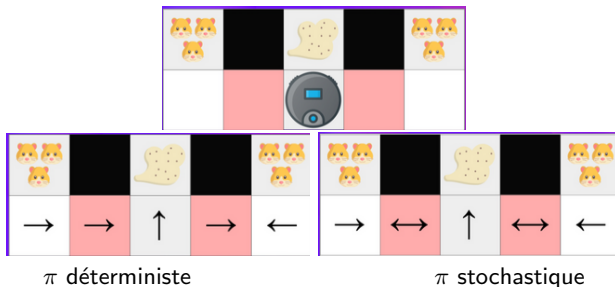
Avec cette politique stochastique, l'action exécutée dans s_0 ne sera pas toujours Right !

Choix d'une politique déterministe ou stochastique

- Selon l'algorithme !
- Les politiques stochastiques gèrent mieux les perceptions ambiguës (*perceptual aliasing*).

Perceptual aliasing

- Observabilité partielle : l'aspirateur perçoit uniquement les murs
- 2 états sont ambiguës et correspondent à la même observation
- une politique stochastique est requise !



Approches basées valeurs versus basées politiques

	Basées Valeurs (type DQN)	Basées politiques
Simplicité	- Fonctions intermédiaires Q	+ Pas de fonctions intermédiaires
Actions	Discrètes	Discrètes et Continues
Exploration	- stratégie spécifique (ϵ -greedy)	+ politiques stochastiques (exploration "apprise")
Stabilité de π	- Oscillations (π calculée à partir de Q , changements brusques)	+ (π modifiée directement)
Efficacité en données	+ (replay buffer, off-policy)	?
Stabilité	+ (variance faible)	?

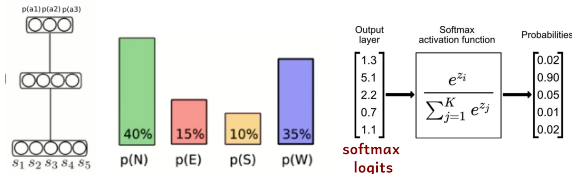
Un algorithme off-policy apprend une politique optimale à partir de données générées par une autre politique, e.g. Q-learning.

- 1 Approches basées valeurs versus basées politiques
- 2 Implémentation des différentes politiques**
- 3 Formalisation du problème de recherche de politique
- 4 Recherche directe de π
- 5 Approches par gradient de π
- 6 Conclusion

Politique paramétrée : implémentation

Actions discrètes : distribution catégorielle

- K actions discrètes
- NN à K sorties : chaque sortie donne la probabilité d'une action ; somme des sorties = 1 (activation softmax)



- Si π est déterministe : l'action exécutée dans s sera celle de plus forte probabilité (argmax)
- Si π est stochastique : l'action exécutée dans s est échantillonnée selon la distribution catégorielle (généralisation loi de Bernouilli)

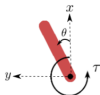
```
m = Categorical(torch.tensor([ 0.25, 0.25, 0.25, 0.25 ]))
m.sample() # échantillonnage de l'action
```

Politique paramétrée : implémentation

Actions continues, politique déterministe

La politique paramétrée renvoie directement l'action à réaliser :

- K actions continues, K sorties
- borner la valeur des sorties si besoin ($\tanh$, ...)



- x, y : cartesian coordinates of the pendulum's end in meters.
- θ : angle in radians.
- τ : torque in $N \cdot m$. Defined as positive counter-clockwise.

Action Space

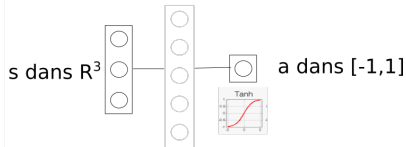
The action is a `ndarray` with shape `(1,)` representing the torque applied to free end of the pendulum.

Num	Action	Min	Max
0	Torque	-2.0	2.0

Observation Space

The observation is a `ndarray` with shape `(3,)` representing the x, y coordinates of the pendulum's free end and its angular velocity.

Num	Observation	Min	Max
0	$x = \cos(\theta)$	-1.0	1.0
1	$y = \sin(\theta)$	-1.0	1.0
2	Angular Velocity	-8.0	8.0

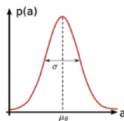


Politique paramétrée : implémentation

Actions continues, politique stochastique : distribution Normale

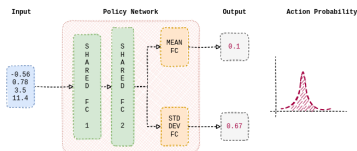
- loi Normale ($\mathcal{N}(\mu, \sigma^2)$) avec moyenne μ et écart-type σ
- la politique (NN) renvoie la moyenne de chaque action
- σ est un facteur d'exploration, soit :
 - σ constant ne dépend pas de s . L'action choisie est : $a = \mu_\theta(s) + \sigma\epsilon, \epsilon \sim \mathcal{N}(0, 1)$
 - la politique renvoie $\log(\sigma)$ et on récupère $\sigma(s) = \exp(\log\sigma(s))$. (ajustement du niveau d'exploration selon l'entrée de la politique)

Policy representation (1D continuous action case)



The stochastic policy is represented as a Gaussian:

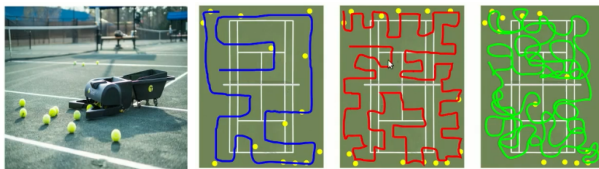
$$\pi_\theta(a_t|s_t) = e^{-\frac{1}{2} \frac{(\mu_\theta - a_t)^2}{\sigma^2}}$$



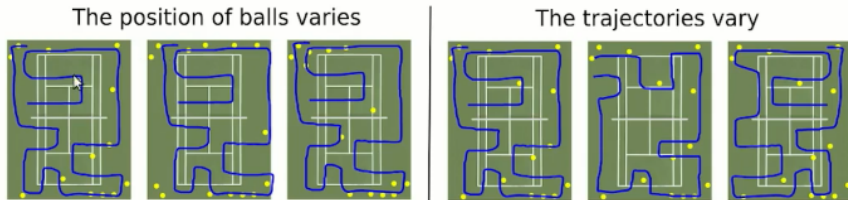
Echantillonnage d'une action de moyenne *mean* et écart-type *std* :

```
#version numpy
action = mean + np.random.randn(1) * std
#version PyTorch
m = Normal(torch.tensor([mean]), torch.tensor([std])) #loi normale N(mean, std)
m.sample()
```


- 1 Approches basées valeurs versus basées politiques
- 2 Implémentation des différentes politiques
- 3 Formalisation du problème de recherche de politique**
- 4 Recherche directe de π
- 5 Approches par gradient de π
- 6 Conclusion

Exemple¹

- Mesure de performance : nombre de balles ramassées dans un temps donné
- Pas de capteurs pour les balles ... se balader + ou - aléatoirement
- Paramètres de la politique : probabilité de tourner à chaque pas, vitesse de déplacement
- Objectif : trouver les paramètres qui maximisent la performance

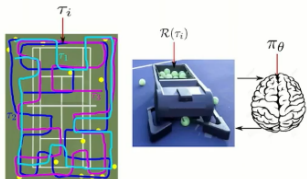
Exemple²

Variations de performance :

- position des balles (stochasticité de l'environnement)
- trajectoire (stochasticité de la politique)

La performance varie beaucoup → besoin de beaucoup de trajectoires pour évaluer une politique

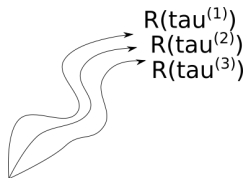
Formalisation du problème de recherche de politique



- π_θ politique paramétrée
- τ_i une trajectoire : séquence d'états-actions sur un horizon H : $\tau = s_0, a_0, s_1, \dots, s_H, a_H, s_{H+1}$
- $R(\tau_i)$ somme des récompense (pondérées) obtenues sur τ_i : $R(\tau) = \sum_{t=0}^H \gamma^t r_t$

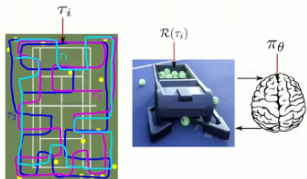
Pour évaluer une politique π_θ on définit la fonction objectif

$$J(\theta) = E_{\tau \sim \pi_\theta} [R(\tau)] = \sum_{\tau} P(\tau|\theta) R(\tau)$$



- pour une même politique π_θ , différentes trajectoires τ sont possibles : $\tau \sim \pi_\theta$
- $P(\tau|\theta)$ probabilité de prendre la trajectoire τ en suivant la politique π_θ
- $R(\tau)$ *performance* obtenue sur une trajectoire τ
- on calcule l'espérance ($E_{\tau \sim \pi_\theta}$) en sommant la *performance* obtenue sur toutes les trajectoires possibles, pondérée par la probabilité de chaque trajectoire

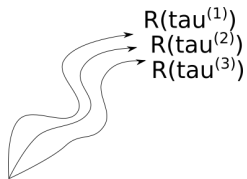
Formalisation du problème de recherche de politique



- π_θ politique paramétrée
- τ_i une trajectoire : séquence d'états-actions sur un horizon H : $\tau = s_0, a_0, s_1, \dots, s_H, a_H, s_{H+1}$
- $R(\tau_i)$ somme des récompense (pondérées) obtenues sur τ_i : $R(\tau) = \sum_{t=0}^H \gamma^t r_t$

Pour évaluer une politique π_θ on définit la fonction objectif

$$J(\theta) = E_{\tau \sim \pi_\theta} [R(\tau)] = \sum_{\tau} P(\tau|\theta) R(\tau)$$



Objectif

Trouver les paramètres θ^* pour que π_θ choisisse des actions qui maximisent la fonction objectif J

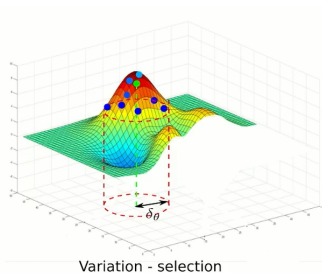
$$\theta^* = \operatorname{argmax}_{\theta} J(\theta)$$

2 approches

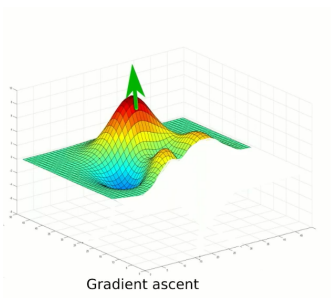
Objectif

Trouver les paramètres θ^* pour que π_θ choisisse des actions qui maximisent la fonction objectif $J : \theta^* = \operatorname{argmax}_\theta J(\theta)$

Recherche directe de Politique



Remontée de gradient



- Approche boîte noire
- Optimisation de θ en maximisant des approximations locales de $J(\theta)$

- Optimisation de θ avec remontée de gradient sur $J(\theta)$

$$\theta_{k+1} = \theta_k + \alpha \nabla_\theta J(\theta)$$

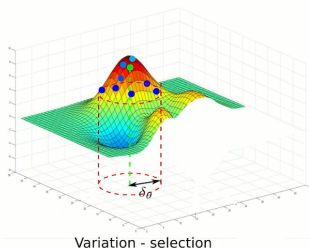
- 1 Approches basées valeurs versus basées politiques
- 2 Implémentation des différentes politiques
- 3 Formalisation du problème de recherche de politique
- 4 Recherche directe de π**
- 5 Approches par gradient de π
- 6 Conclusion

Principe général

Objectif

Trouver les paramètres θ^* pour que π_θ choisisse des actions qui maximisent la fonction objectif J : $\theta^* = \operatorname{argmax}_\theta J(\theta) = \operatorname{argmax}_\theta E_{\tau \sim \pi_\theta} [R(\tau)]$

- Moyen : Optimisation boîte noire : l'unique hypothèse est de pouvoir **estimer** J pour toute valeur de θ
- Principe : exploration de l'espace des paramètres θ (donc espace des politiques π_θ)



- $J(\theta)$ de forme inconnue
- Choisit une valeur de départ pour θ
- Perturbations aléatoires des paramètres θ pour générer des π candidates
- Evaluation des π candidates par interaction avec l'environnement (estimation de J)
- Conservation des meilleures π

Variations entre algorithmes d'optimisation : comment explorer l'espace des paramètres θ

Algo 1 : Hill Climbing

Trouver un optimum local parmi un ensemble de *configurations*.

- configuration : valeur de θ
- fonction-objectif pour évaluer chaque configuration : $G = R(\tau)$
somme des récompense sur une trajectoire, estimation *imparfaite* de J .
- génération de *configuration* voisine : bruit sur $\theta \rightarrow \pi$ candidates

Hill Climbing

Initialize the weights θ in the policy arbitrarily.

Collect an episode with θ , and record the return G .

$\theta_{best} \leftarrow \theta, G_{best} \leftarrow G$

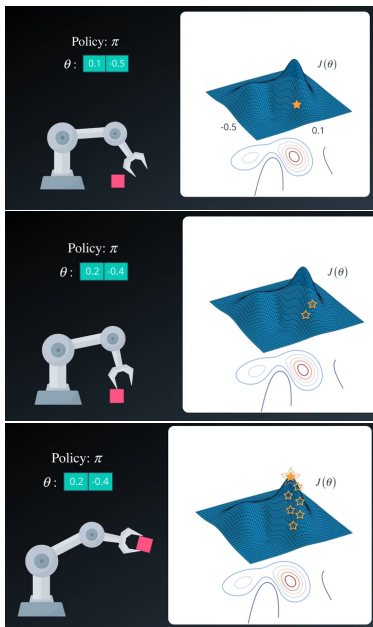
Repeat until environment solved:

Add a little bit of random noise to θ_{best} , to get a new set of weights θ_{new} .

Collect an episode with θ_{new} , and record the return G_{new} .

If $G_{new} > G_{best}$, then:

$\theta_{best} \leftarrow \theta_{new}, G_{best} \leftarrow G_{new}$



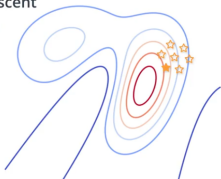
Autres algos

Hill Climbing n'est pas assuré de converger vers une politique optimale.

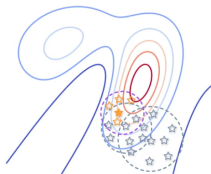
Améliorations directes de Hill Climbing

- Steepest Ascent Hill Climbing : génère **plusieurs** π **candidates** à chaque itération et conserve la meilleure. Réduit le risque de rester bloqué dans une solution sous-optimale.
- Recuit simulé : réduction de la **variance du bruit** utilisé pour générer les π candidates.
- Adaptive Noise Scaling : la **variance du bruit** est réduite si on trouve une meilleure π candidate, augmentée sinon.
- Cross Entropy Method : sélectionne plusieurs meilleures π parmi les candidates et prend la moyenne.

Steepest Ascent
Hill Climbing



Simulated
Annealing



- 1 Approches basées valeurs versus basées politiques
- 2 Implémentation des différentes politiques
- 3 Formalisation du problème de recherche de politique
- 4 Recherche directe de π
- 5 Approches par gradient de π
- 6 Conclusion

Principe général

Objectif

Trouver les paramètres θ^* pour que π_θ choisisse des actions qui maximisent la fonction objectif J : $\theta^* = \operatorname{argmax}_\theta J(\theta) = \operatorname{argmax}_\theta E_{\tau \sim \pi_\theta} [R(\tau)]$

- Principe : Pour une trajectoire τ qui a donné une bonne performance $R(\tau)$, on va **augmenter la probabilité de choisir les actions prises dans cette trajectoire**.

Training Loop:

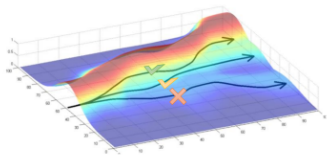
Collect an **episode with the π** (policy).

Calculate the return (sum of rewards).

Update the weights of the π :

If **positive return** → **increase** the probability of each (state, action) pairs taken during the episode.

If **negative return** → **decrease** the probability of each (state, action) taken during the episode



- Moyen : Amélioration de la politique avec remontée de gradient :

$$\theta_{k+1} = \theta_k + \alpha \nabla_\theta J(\theta)$$

- Calcul de $\nabla_\theta J(\theta)$ avec Policy Gradient theorem : algorithme REINFORCE (1992) premier algorithme de RL basé sur le gradient de la politique avec une approche Monte-Carlo.

- 1 Approches basées valeurs versus basées politiques
- 2 Implémentation des différentes politiques
- 3 Formalisation du problème de recherche de politique
- 4 Recherche directe de π
- 5 Approches par gradient de π
- 6 Conclusion

Approches basées Valeurs vs basées Politiques

	Basées Valeurs (type DQN)	Basées politiques (type Reinforce)
Simplicité	- Fonctions intermédiaires Q	+ Pas de fonctions intermédiaires
Actions	Discrètes	Discrètes et Continues
Exploration	- stratégie spécifique (ϵ -greedy)	+ politiques stochastiques (exploration "apprise")
Stabilité de π	- Oscillations (π calculée à partir de Q , changements brusques)	+ (π modifiée directement)
Efficacité en données	+ (replay buffer, off-policy)	- (pas de replay buffer)
Stabilité	+ (variance faible)	- (variance élevée (monte-carlo))

Les approches Acteur-Critic (type **PPO**) combinent les 2 : ajustement des probabilités des actions de l'acteur, mais utilisation d'un critic pour évaluer plus rapidement les bonnes et mauvaises actions prises...

Large variétés d'algorithmes

Table 1. Table summary of model-free RL algorithms.

Algorithm	Agent Type	Policy	Policy Type	MC or TD	Action Space	State Space
State action reward state action (SARSA) SARSA Lambda	Value-based	On-policy	Pseudo-deterministic ($\epsilon - greedy$)	TD	Discrete only	Discrete only
Deep Q Network (DQN) Double DQN Noisy DQN Prioritized Replay DQN Dueling DQN Categorical DQN Disturbed DQN (C51)	Value-based	Off-policy	Pseudo-deterministic ($\epsilon - greedy$)		Discrete only	Discrete or Continuous
Normalized Advantage Functions (NAF) = Continuous DQN	Value-based				Continuous	Continuous
REINFORCE (Vanilla policy gradient)	Policy-based	On-policy	Stochastic	MC		
Policy Gradient	Policy-based		Stochastic			
TRPO	Actor-critic	On-policy	Stochastic		Discrete or Continuous	Discrete or Continuous
PPO	Actor-critic	On-policy	Stochastic		Discrete or Continuous	Discrete or Continuous
A2C/A3C	Actor-critic	On-policy	Stochastic	TD	Discrete or Continuous	Discrete or Continuous
DDPG	Actor-critic	Off-policy	Deterministic		Continuous	Discrete or Continuous
TD3	Actor-critic				Continuous	Discrete or Continuous
SAC	Actor-critic	Off-policy			Continuous	Discrete or Continuous
ACER	Actor-critic				Discrete	Discrete or Continuous
ACKTR	Actor-critic				Discrete or Continuous	Discrete or Continuous

Tableau incomplet ...

Biblio

Pour en savoir plus, voir :

- [Reinforcement Learning Virtual School](#)
- [Spinning Up OpenAI](#)
- [DeepRL Bootcamp Lecture](#)